StatSumMediatedCapacity [SimTalk]

Returns the sum of the brokered capacity of the *Exporter* designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>.StatSumMediatedCapacity → integer
```

Return Value

The return value has the data type *integer*.

Example

```
print MyExporter.StatSumMediatedCapacity
```

See also

Tab Statistics [Exporter]

Statistics report, **Service Statistics — States — Capacities and States of the Exporters**

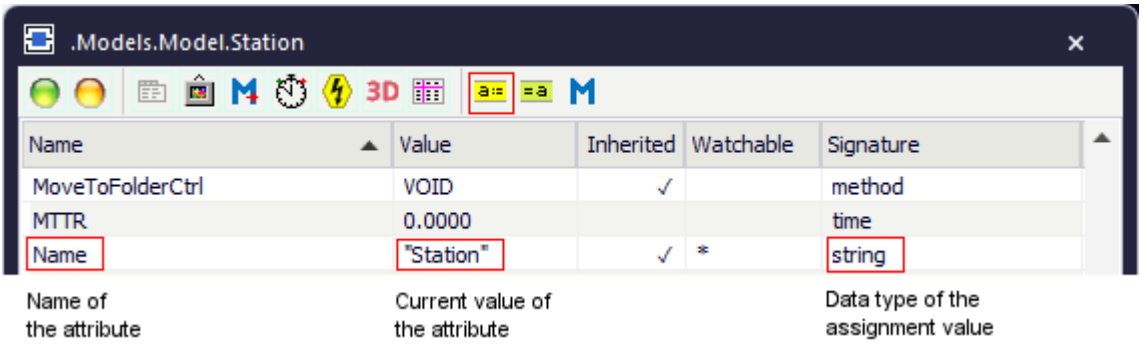
Attributes of the Exporter

_Attributes of the Exporter

The *Exporter* provides:

- The *attributes* listed in the table of contents to the left.
- The **Attributes of All Objects**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.

You can set the value of an attribute and you can get its value, either with the check boxes, the text boxes and drop-down lists in the dialog windows or by assigning values to the respective attributes.

- To **set the value** of an attribute, you might, for example, type:

```
MyExporter.ExpStatOn := true
```

- To **get the value** of an attribute, you might, for example, type:

```
print MyExporter.ExpStatOn
posit := Station.Cont.XPos
```

AutomaticMediation [SimTalk] - Exporter

Prevents that the *Exporter* designated by <Path> is automatically assigned by the *Broker*.

Type

Attribute

Syntax

```
<Path>.AutomaticMediation:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Specify `true` to make the *Broker* assign the *Exporter* automatically.

Specify `false` to assign the *Exporter* yourself.

Examples

```
.Resources.myExporter:2.AutomaticMediation := true
```

```
param type: integer // Importer type (0=failure, 1=setup, 2=processing,
3=transport)
var t : table
switch type
case 0
   ?.failImp.releaseExporters
case 1
   ?.setupImp.getExporters(t)
   if t.yDim > 0
       t[1,1].AutomaticMediation := true
   end
   ?.setupImp.releaseExporters
case 2
   ?.imp.getExporters(t)
   if t.yDim > 0
       t[1,1].AutomaticMediation := true
   end
   ?.imp.releaseExporters
end
```

SimTalk

importExporter [SimTalk]

BrokerPath [SimTalk] - Exporter

Sets the path to the *Broker* which procures the services that the *Exporter* designated by <Path> provides.

Type

Attribute

Syntax

```
<Path>.BrokerPath:object
```

Assignment Value

You can assign a value of data type *object*.

Example

```
print MyExporter.BrokerPath
MyExporter.BrokerPath := myBroker
```

See also

Broker [Exporter]

Capacity [SimTalk] - Exporter

Sets the **Capacity** of the *Exporter* designated by <Path>.

Remarks

The *Capacity* is a value greater than or equal to zero.

> **Note**
> You can only reduce the capacity if the **Capacity** provided to the *importers* is not fallen short of. If you increase the capacity, the *Broker* will immediately attempt to find new *importers*.

Type

Attribute

Syntax

```
<Path>.Capacity:integer
```

Assignment Value

You can assign a value of data type *integer*.

Example

```
MyExporter.Capacity := 2
```

See also

Capacity [text box] - Exporter

ExpStatOn [SimTalk]

Activates exporter statistics of the *Exporter* designated by <Path> (`true`) or deactivates it (`false`).

Type

Attribute

Syntax

```
<Path>.ExpStatOn:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyExporter.ExpStatOn := true
```

See also

Exporter Statistics [Exporter]

FailServices [SimTalk]

Fails the services (`true`) which the *Exporter* designated by <Path> provides (`true`) or does not fail them (`false`).

Remarks

If you set *FailServices* to `false` and the *Exporter* provides services for an *importer*, *Plant Simulation* does not interrupt the respective process of the *importer* at the beginning of a failure. The respective **Out** event thus persists. During a failure new requests to the *Exporter* remain unsatisfied.

If *Plant Simulation* interrupts processing, for example through a failure of the *Exporter*, *Plant Simulation* deletes the respective **Out** event and adds it anew at the end of the failure according to the remaining processing time.

If you set *FailServices* to `true` and the *Exporter* provides services for an *importer*, *Plant Simulation* interrupts the respective process for the duration of the failure.

Note

The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

In *SimTalk* you can compare strings in an case-insensitive manner with the **~=** operator, compare **Relational Operators**.

Type

Attribute

Syntax

```
<Path>.FailServices:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyExporter.FailServices := true
```

See also

Fail Services [text box]

Relational Operators

OrderCtrl [SimTalk] - Exporter

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation calls the *Method* whenever the *Exporter* is assigned to an *importer*.

It determines how the *Exporter* handles the order.

Type

Attribute

Syntax

```
<Path>.OrderCtrl:method
```

Parameters

The **Order Control** has two parameters that define the *importer*:

- The parameter *Importer* of data type *object* designates the *importer*.
- The parameter *Type* of data type *integer* designates its **type**: `0` designates the *failure/remove failure-importer*, `1` the *set-up-importer*, `2` the *processing-importer*, and `3` the *transport-importer*.

Reformatting the Method

If you manually enter an **Order Control** and click **Apply** or **OK**, *Plant Simulation* automatically checks if the *Method* expects the correct parameters. If this is not the case, *Plant Simulation* shows a message asking if the *Method* shall be reformatted or not. Empty *methods* are reformatted automatically. You cannot suppress checking of the format of the *Method*.

Assignment Value

You can assign a value of data type *method*.

Example

```
MyExporter.OrderCtrl := &myOrderCtrl
```

See also

Order Control [Exporter]

Priority [SimTalk] - Exporter

Sets the **Priority** with which the *Exporter* designated by <Path> exports services when fulfilling a request.

Remarks

The *Priority* of an *Exporter* is a criterion for the urgency of a request. The *Priority* is an integer value. The higher its value, the higher the urgency. The **Priority** only applies to *Exporters* who are registered with the same *Broker*.

If several qualified *Exporters* with the same priority exist, who can provide the requested service, *Plant Simulation* first brokers the *Exporters*, who already stay on a suitable *Workplace* at the station. After that *Exporters* are brokered who already are at the station, but stay on the wrong *Workplace*. Then *Exporters* are brokered who are not staying on a *Workplace*. Finally, those *Exporters* are brokered who stay on a *Workplace* that is assigned to another station.

Type

Attribute

Syntax

```
<Path>.Priority:integer
```

Assignment Value

You can assign a value of data type *integer*.

Exporters with a higher capacity will be brokered earlier by the *Broker* for identical services than *Exporters* with a lower priority.

Plant Simulation provides an *Exporter* with *Priority* 10 before providing an *Exporter* with *Priority* 1.

Example

```
MyExporter.Priority := 3
```

See also

Priority [Exporter]

ReleaseCtrl [SimTalk] - Exporter

Designates a *Method* object of the object designated by <Path>.

Remarks

Plant Simulation executes the *control* as soon as any *importer* releases the *Exporter*.

Type

Attribute

Syntax

```
<Path>.ReleaseCtrl:method
```

Parameters

The **Release Control** has two parameters that characterize the *importer*:

- The parameter *Importer* of data type *object* designates the *importer* proper.
- The parameter *Type* of data type *integer* designates its **type**: `0` designates the *failure/remove failure-importer*, `1` the *set-up-importer*, `2` the *processing-importer*, and `3` the *transport-importer*.

When the control is called, the *Exporter* has already left its *importer*. If you entered a *Method*, you yourself have to take care that the *Exporter* is assigned new *importers*, for example with the method *findNewImporter*. In this case the *Exporter* will not automatically register as being available with its *Broker*.

Reformatting the Method

If you manually enter an **Release Control** and click **Apply** or **OK**, *Plant Simulation* automatically checks if the *Method* expects the correct parameters. If this is not the case, *Plant Simulation* shows a message asking if the *Method* shall be reformatted or not. Empty *methods* are reformatted automatically. You cannot suppress checking of the format of the *Method*.

Assignment Value

You can assign a value of data type *method*.

Example

```
MyExporter.ReleaseCtrl := &myReleaseCtrl
```

SimTalk

findNewImporter [SimTalk]

See also

Release Control [Exporter]

Services [SimTalk] - Exporter

Sets or returns the names of the **Services** that the *Exporter* designated by <Path> exports.

Remarks

The name is not case-sensitive, just like the names of *attributes* and *methods* of the objects are not case-sensitive.

To save memory and improve access speed, all places which are using such a case-insensitive string are pointing to the same string in main memory. The visible and unexpected result is that the first occurrence of the string defines how the string is written in terms of upper- and lower-casing.

In *SimTalk* you can compare strings in an case-insensitive manner with the **~=** operator, compare **Relational Operators**.

Type

Attribute

Syntax

```
<Path>.Services:array[]
```

Assignment Value

You can assign an *array* of data type *string*.

Examples

```
var a : string[] := ["Job1", "Job2", "Job3"]
Exporter2.Services := a
```

```
print MyExporter.Services
```

SimTalk

hasService [SimTalk]

See also

Services [Exporter]

Exported Services [Exporter]

Relational Operators

Broker

Broker [object]

Use the object *Broker* for brokering offered services and required services. You might model the **manager** of a **plant**, the **supervisor** of a **department**, or the **foreman** of a **shop** with it.

Image

Remarks

The *Broker* cooperates with the **Exporter** and the *importers* (see **Tab Importer** and **Sub-tab Failure**) of the *Station*, the *ParallelStation*, the *AssemblyStation*, the *DismantleStation*, the *DePortioner*, the *Mixer*, the *Portioner*, and the *Tank*.

The *Broker* is the go-between for services offered and services required. You might model the manager of a **plant**, the **supervisor** of a department, or the **foreman** of a shop with it. Each *Broker* can manage several *Exporters*/*Workers*, which tender services, and it may receive requests from several *importers* that require services. A request consists of a list of the required services and the amount of required services. A service name is a *string*.

When a *Broker* receives a request from an *importer*, it immediately attempts to fulfill it using the *Exporters*/*Workers* it manages. If the *Broker* does not succeed in doing this, it may also pass the request on to other *Brokers* with which it is connected with a *Connector*. The direction of the *Connector* determines the direction in which *Plant Simulation* passes the request on. If the *Exporters*/*Workers* of different *Brokers* are able to satisfy the request, those *Exporters*/*Workers* are assigned. If the request cannot be satisfied immediately, the *Broker* that received it first will save it.

The *Broker* then attempts to provide the service at a later point in time. To do this, he goes from requested service to requested service and attempts to find an *Exporter*/*Worker* for each. If an *Exporter*/*Worker* offers the service, it is going to reserve the maximum possible number or the necessary number. Then the *Exporter*/*Worker* cannot offer this reserved amount of services for other services. For this reason we recommend to request special services first and more general ones last.

> **Note**
>
> If the *importer* requests several services at the same time, all of these services have to be available at the same time, before the *Broker* assigns them to the station.

The *Broker* does not run any optimization! That way it may happen that the *Broker* cannot assign any *Exporters*/*Workers*, although the *Exporters*/*Workers* may theoretically be assigned.